

Project Asphalt : A Native Assembly(8088) Arcade Racer

with Interrupt Driven Audio



Muhammad Obaidullah (24L-0509)

Muhammad Usman Rafique (24L-0657)

National University of Computer and Emerging Sciences Lahore Campus

EE2003: COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

Abstract

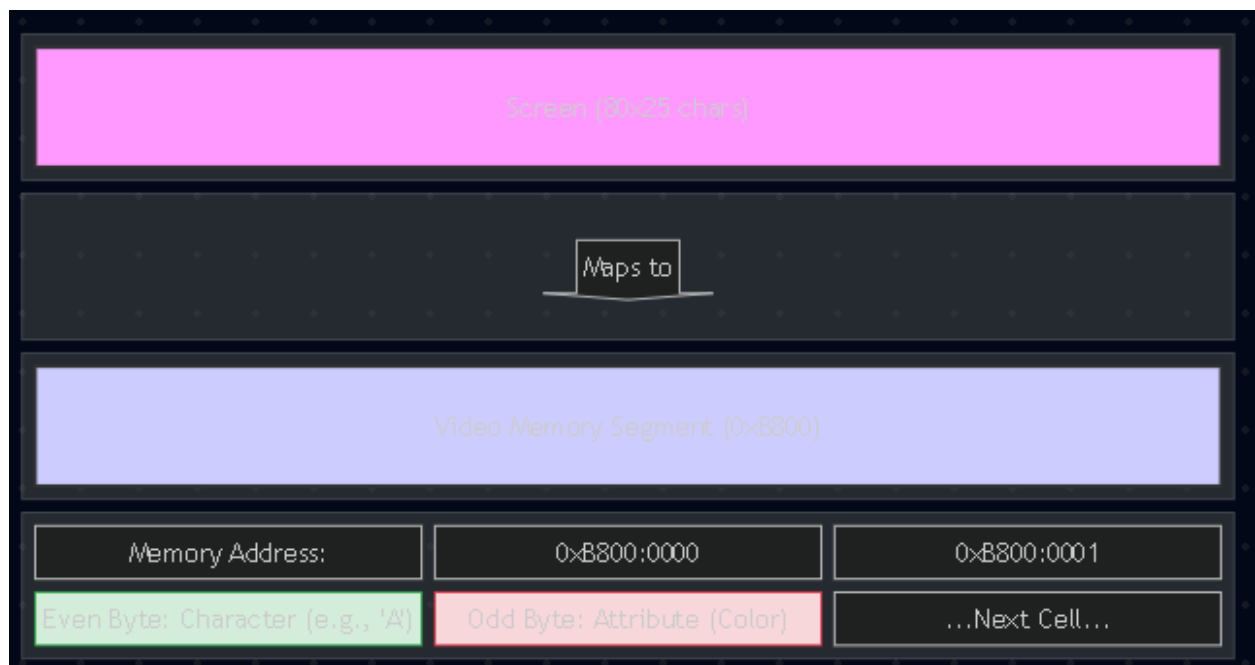
"Asphalt" is a real-time arcade racing simulation developed in **8088 Assembly Language**. The primary objective of this project was to demonstrate low-level system control, direct hardware manipulation, and efficient memory management within the constraints of the 16-bit Real Mode environment. Unlike high-level abstractions, this project interacts directly with the CPU registers, video memory buffers, and hardware I/O ports to deliver a fluid gaming experience.

Furthermore, the application utilizes Direct Memory Access (DMA) to the Video Segment (0xB800) for rendering, bypassing standard BIOS interrupts to achieve fluid frame rates. The result is a fully functional, interactive game engine featuring collision detection, dynamic resource management (fuel, economy), and pseudo-random procedural generating, proving that complex, responsive software can be engineered with minimal system overhead.

1. System Architecture & Memory Model

The application is designed as .COM executable, operating within a single 64KB segment for code, data, and stack, ensuring minimal overhead.

1.1. Video Memory Direct Access (DMA)



Standard BIOS interrupts (like INT 10h) are too slow for real-time game loops due to the overhead context switching. Asphalt utilizes **Direct Memory Access** to the VGA Video Segment at address 0xB800.

- **Mapping Logic:** The screen is treated as a linear array of 4000 bytes (80 columns $\times$ 25 rows $\times$ 2 bytes per cell).
- **Addressing Formula:** The offset for any coordinate (row, column) is calculated as:

$$\text{Offset} = (\text{Row} \times 160) + (\text{Column} \times 2)$$

- **Attribute Byte:** The even byte holds the ASCII character, and the odd byte holds the color attribute. This allows for the custom design of the car sprites and road textures without loading external assets.

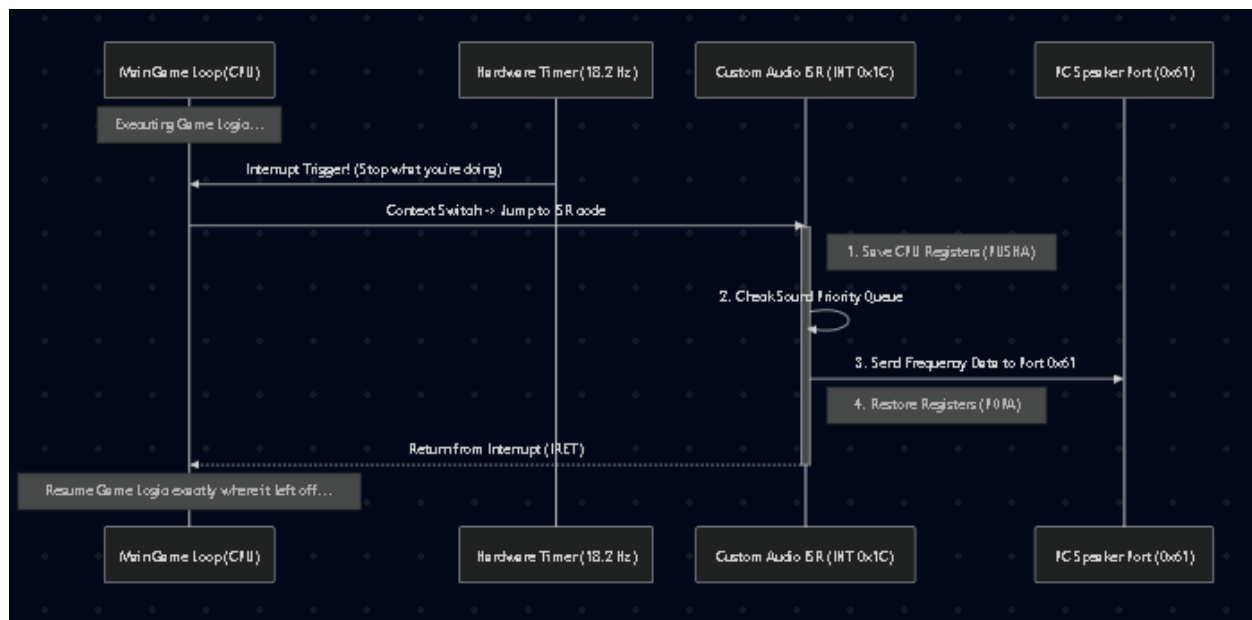
1.2. The Game Loop

The core of the application relies on a polling-based game loop (main loop) that cycles through three phases:

1. **Input Handling:** Checks the keyboard buffer via INT 16h.
2. **Logic Update:** Calculates collisions, fuel consumption, and object movement.
3. **Rendering:** Refreshes the 0XB800 buffer to display changes.

2. Key Technical Implementations

2.1. Multitasking via Custom Interrupt Service Routine (ISR)



The most significant technical achievement in this project is the implementation of a non-blocking audio engine.

- **The Problem:** Standard synchronous execution pauses the CPU while a sound plays, freezing the game.
- **The Solution (Hooking 0x1C):** The code hooks the Timer Interrupt (INT 1Ch), which triggers approximately 18.2 times per second.
- **Logic Flow:**
 1. The original ISR address is saved to `oldisr`.
 2. The Instruction Pointer (IP) for 0x1C is redirected to the custom `timer_isr` label.
 3. Inside `timer_isr`, the code checks a priority queue: **Collision Sound > Coin Sound > Background Music**.
 4. The hardware speaker is manipulated directly via **Port 0x61** (Speaker Control) and **Port 0x42/0x43** (PIT Timer 2), allowing sound frequencies to change between frame renders without pausing the game logic.

2.2. Pseudo-Random Number Generation (PRNG)

To ensure dynamic gameplay, obstacles and coins spawn in unpredictable lanes. Since 8088 lacks a hardware random number generator, a software solution was implemented:

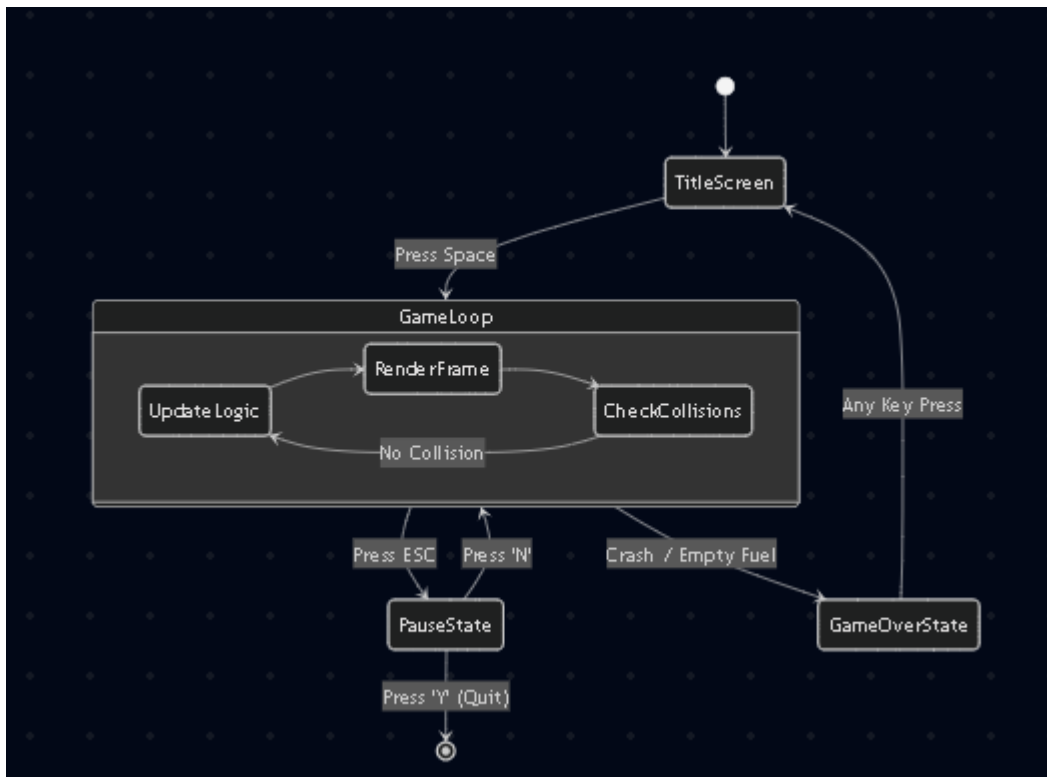
- **Source of Entropy:** The System Time is read using INT 1A.
- **Algorithm:** The lower bits of the clock tick count (DX register) are used as the seed. This value is divided by the `max_random` constraint (e.g., 3 lanes), and the remainder (DL) determines the spawn position.

2.3. Collision Detection Engine:

Collision is handled using AABB (Axis-Aligned Bounding Box) logic.

- **Lane Logic:** The code checks if `player_col == obstacle_col`.
- **Proximity Logic:** If the columns match, the code calculates the delta between `player_row` and `obstacle_row`. A collision is registered only if the delta falls within the vertical height of the sprite (approx. 4 rows).
- **Optimization:** To save cycles, collision checks are skipped if the object is currently inactive (`timer = 0`).

3. Game Mechanics & State Management



3.1. Resource Economy (Fuel System)

To prevent passive play, a decaying resource (Fuel) was introduced.

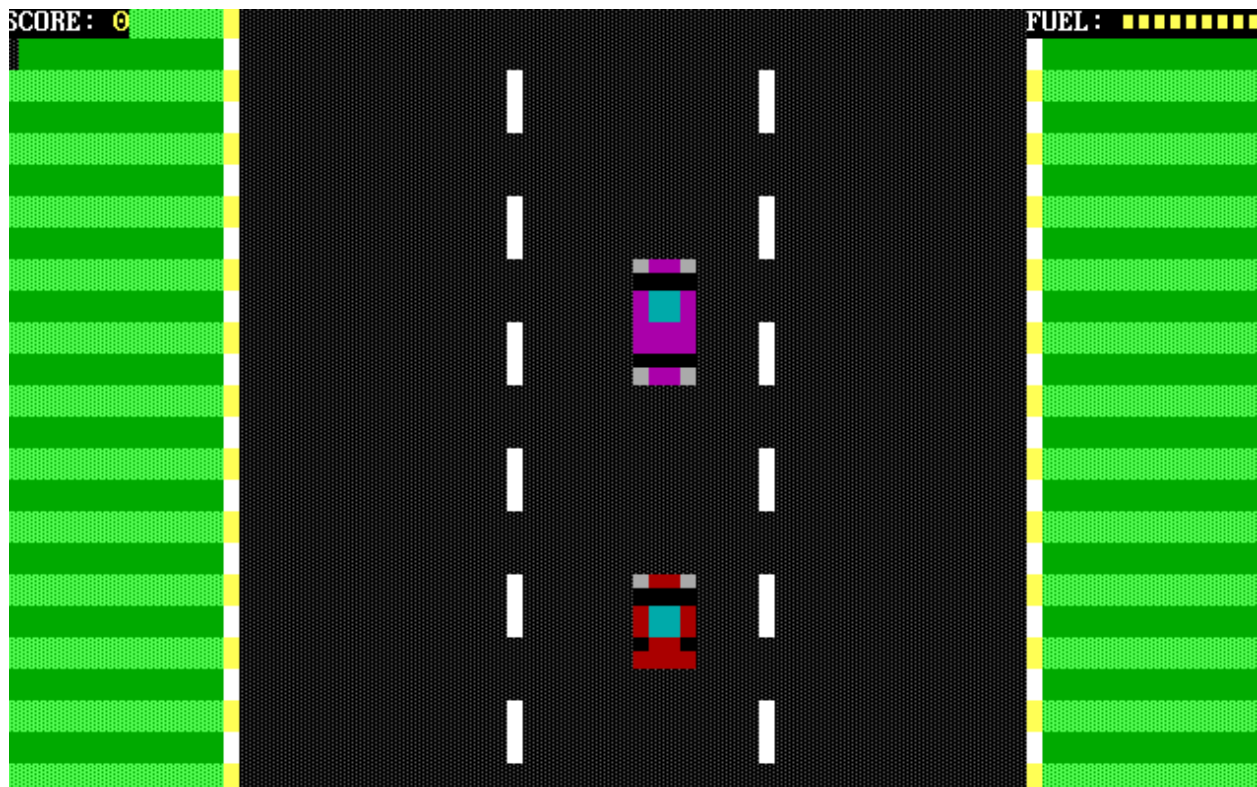
- **Decay Rate:** Fuel decreases every 36 frames.

- **Replenishment:** Collecting a “Coin” object increments the score and adds 5 units to the Fuel variable.
- **Failure State:** The game terminates if Fuel reaches 0 or a collision occurs, triggering the `game_over_screen`.

3.2. Sprite Design

Sprites (Player Car, Obstacles, Coins) are not image files but are constructed programmatically using extended ASCII characters:

- **Tires:** Character 0xDC (Lower Half Block) with attribute 0x70 (Black on Grey).
- **Body:** Character 0xDB (Full Block) with attribute 0x04 (Red) for the player and 0x05 (Magenta) for enemies.



4. Challenges and Solutions

Challenge	Solution
Blocking Audio	Implemented a custom ISR to handle sound processing in the background, allowing the main loop to run interrupted.
Screen Flicker	Used double buffering logic implicitly writing to Video Memory faster than the refresh rate and optimizing the drawBackground routine to only redraw necessary segments.
Register Corruption	Extensive use of PUSH and POP (or PUSHA/POPA) at the start and end of every subroutine to preserve the state of general-purpose registers.
Randomness	The INT 1A method initially produced patterns; added a "shuffle" logic by changing max_random dynamically to vary the spawning intervals.

5. Future Enhancements:

- **Score Persistence:** Implementation of file I/O (INT 21h) to save high scores to a .txt file.
- **Difficulty Scaling:** Automatically increasing the game speed (delay loop reduction) as the score increases.
- **Two-Player Mode:** Utilizing the split-screen capability of the 80x25 text mode to allow local multiplayer on a single keyboard.

6. Conclusion:

Project Asphalt successfully demonstrates the power of low-level programming. By bypassing operating system abstractions, the project achieves high-speed rendering and multitasking on simulated 16-bit hardware. The implementation of the custom ISR for audio specifically highlights a mastery of hardware interrupts and system architecture.